The Harness Engineer's Guidebook

A Comprehensive Guide to Building Production-Grade AI Agent Infrastructure

Harness Engineering Team

May 10, 2026

Contents

1 The Foundations of Harness Engineering **1**
1.1 The Core Philosophy 1
1.2 The 5 Pillars of an Agent Harness 1

2 Guidebook for Further AI Agent Study **3**
2.1 Advanced Memory Architectures 3
2.2 Multi-Agent Orchestration 3
2.3 Algorithmic Prompt Optimization 3
2.4 Security and Sandboxing 4

3 Phase 1: Foundations (The Stateless Phase) **5**
3.1 Step 1: API Mastery 5
3.2 Step 2: Structured Outputs 6
3.3 Step 3: The ReAct Loop 8

4 Phase 2: ACI and Tool Engineering (The Hands Phase) **10**
4.1 Step 4: Safe SQL Tool 10
4.2 Step 6: Error-Handling Refinement 11

5 Phase 3: Memory and Orchestration (The Brain Phase) **13**
5.1 Step 7 & 8: RAG and Context Compaction 13
5.2 Step 9: State Machines 14

6 Phase 4: Production and Safety (The Harness Phase) **16**
6.1 Step 10: Sandboxed Code Execution 16
6.2 Step 11: Evaluation Suite 17
6.3 Step 12: Human-in-the-Loop (HITL) 18

Chapter 1

The Foundations of Harness Engineering

1.1 The Core Philosophy

The fundamental principle of modern AI development is encapsulated in the following equation:

$$\textbf{Agent} = \textbf{Model} + \textbf{Harness}$$

An LLM alone is just a stateless text-prediction engine. *Harness Engineering* is the discipline of building the surrounding software infrastructure (the "harness") that gives the model memory, tools, boundaries, and autonomy. Instead of relying on "prompt engineering" to fix failures, a Harness Engineer builds system-level solutions to prevent them entirely.

1.2 The 5 Pillars of an Agent Harness

A production-grade agent is built upon five essential pillars that provide the necessary infrastructure for reliable and autonomous operation.

1. **Context Management (The Brain's Workspace):** This pillar involves the dynamic assembly of prompts, history compaction, and memory management. Its primary goal is to provide the model with the most relevant information while ensuring the context window does not overflow.

2. **Cognitive Framework:** This defines the agent's worldview, mission, and behavioral boundaries. Often codified in files like `agents.md`, it serves as a map for the agent, directing it on how to find information and how to behave, rather than being a static encyclopedia of knowledge.

3. **Agent-Computer Interface (ACI):** Much like a User Interface (UI) is designed for humans, the ACI is designed for agents. It focuses on structured JSON-in/JSON-out tool design. The ACI follows a clear hierarchy:

   - **Context/Maps:** Data dictionaries, documentation, and SOPs.
   - **Observation:** Read-only access to time-series, relational databases, or logs.

- **Compute:** Sandboxed execution environments (e.g., Python kernels, WASM) for data processing.
- **Action:** Systems of record and communication (e.g., Jira, Slack, API mutations).

4. **Standard Workflows & Refinement Loops:** This pillar enforces structured workflows such as the *Plan-Generate-Evaluate* cycle. It enables the agent to self-correct based on actionable error feedback, such as feeding a Python stack trace back to the LLM when a tool call fails.

5. **Evaluation & Guardrails:** This involves implementing deterministic checks (e.g., linters, static analysis) and using LLM-as-a-judge patterns to verify outputs and behaviors before an agent is permitted to take final actions in a production environment.

Chapter 2

Guidebook for Further AI Agent Study

To deepen your expertise in Harness Engineering, focus on these advanced topics, which represent the current frontier of agentic systems.

2.1 Advanced Memory Architectures

Modern agents require long-term memory that spans multiple sessions without exhausting the context window.

- **Topics:** Vector Databases (Pinecone, Milvus), Graph RAG (Knowledge Graphs), Episodic vs. Semantic Memory.
- **Key Concept:** Treating the context window like RAM and vector databases like a hard drive. Study architectures like **MemGPT**, which autonomously manages its own memory by moving data between these layers.

2.2 Multi-Agent Orchestration

Complex tasks are often handled better by a team of specialized agents rather than a single massive prompt.

- **Topics:** Supervisor models, specialized sub-agents, routing, and consensus algorithms.
- **Key Concept:** Learn to build workflows where a "Planner Agent" delegates to a "Coder Agent," whose output is then verified by a "QA Agent." Frameworks like **LangGraph** and **Microsoft AutoGen** are essential for this study.

2.3 Algorithmic Prompt Optimization

As agentic systems scale, manual prompt tweaking becomes unsustainable.

- **Topics:** LLM-as-a-Judge, Prompt routing, and frameworks like **DSPy**.
- **Key Concept:** Use programmatic optimization to find the best prompts and weights based on validation metrics. This transition moves prompt design from an "art" to a "data science" discipline.

2.4 Security and Sandboxing

Executing agent-generated code or giving agents access to APIs requires rigorous security boundaries.

- **Topics:** WebAssembly (WASM), Ephemeral Docker containers, and Permission Bridges.
- **Key Concept:** Implement sandboxed execution environments and Human-in-the-Loop (HITL) approval gates to prevent destructive actions and data exfiltration.

Chapter 3

Phase 1: Foundations (The Stateless Phase)

In this phase, we focus on understanding how to control Large Language Model (LLM) outputs deterministically. This is the bedrock of Harness Engineering: moving away from "prompt and pray" towards rigorous software control.

3.1 Step 1: API Mastery

The goal of this step is to understand core LLM APIs and parameters. A Harness Engineer must own the model's behavior through deterministic settings.

```python
"""
PHASE 1 - STEP 1: API Mastery
Harness Pillar: Cognitive Framework
Goal: Understand how to deterministically control Claude's output using API parameters.
"""

import anthropic
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic()

# 1a. Basic call - observe the raw structure
response = client.messages.create(
    model="claude-opus-4-5-20251101",
    max_tokens=256,
    messages=[{"role": "user", "content": "What is 17 x 24?"}],
)
print(f"Answer: {response.content[0].text}")

# 1b. Temperature = 0 (deterministic output)
for _ in range(3):
    r = client.messages.create(
        model="claude-opus-4-5-20251101",
        max_tokens=64,
```

```
        temperature=0,
        messages=[{"role": "user", "content": "Give me one word for 'happy'."}],
    )
    print(r.content[0].text.strip())

# 1c. Stop sequences - force early termination
r = client.messages.create(
    model="claude-opus-4-5-20251101",
    max_tokens=256,
    stop_sequences=["STOP"],
    messages=[{"role": "user", "content": "Count from 1 to 10. After each number write 'STOP' on a new line."}],
)
print(r.content[0].text)

# 1d. System prompt - define the Cognitive Framework
SYSTEM = """You are a financial data analyst assistant.
Rules:
1. ONLY answer questions about financial data.
2. ALWAYS cite your reasoning step-by-step.
3. Format all monetary values as: $X,XXX.XX
"""
r = client.messages.create(
    model="claude-opus-4-5-20251101",
    max_tokens=256,
    system=SYSTEM,
    messages=[{"role": "user", "content": "What's the PE ratio of Apple?"}],
)
print(r.content[0].text)
```

Listing 3.1: step1_api_mastery.py

Key Takeaways

- Always set `temperature=0` for deterministic agent actions.
- Use `stop_sequences` to force the model to produce parseable output.
- The system prompt IS your agent's Cognitive Framework—define its identity and boundaries here.

3.2 Step 2: Structured Outputs

Harness Engineers never parse free text with regular expressions if they can avoid it. Instead, they contract the model to emit machine-readable structured data using schemas like Pydantic.

```
"""
PHASE 1 - STEP 2: Structured Outputs
```

```python
Harness Pillar: ACI (Agent-Computer Interface)
Goal: Force Claude to reply in strictly validated JSON schemas using Pydantic.
"""

from typing import Literal
import anthropic
from pydantic import BaseModel, Field
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic()

class Company(BaseModel):
    name: str
    founded_year: int
    headquarters: str
    main_product: str

response = client.messages.parse(
    model="claude-opus-4-5-20251101",
    max_tokens=256,
    output_format=Company,
    messages=[{"role": "user", "content": "Tell me about Anthropic, the AI safety company."}],
)

company: Company = response.content[0].parsed
print(f"Name: {company.name}, Founded: {company.founded_year}")

# Decision schema - agent decision-making
class AgentDecision(BaseModel):
    action: Literal["approve", "reject", "escalate"]
    reason: str
    confidence: float = Field(ge=0.0, le=1.0)

SCENARIO = "User wants to withdraw $50,000. Balance is $52,000. Account is 2 years old."

response = client.messages.parse(
    model="claude-opus-4-5-20251101",
    max_tokens=256,
    system="You are a bank fraud detection agent.",
    output_format=AgentDecision,
    messages=[{"role": "user", "content": f"Evaluate: {SCENARIO}"}],
)

decision: AgentDecision = response.content[0].parsed
print(f"Action: {decision.action}, Confidence: {decision.confidence:.0%}")
```

Listing 3.2: step2_structured_outputs.py

Key Takeaways

- `client.messages.parse()` is the cleanest pattern for structured outputs.
- Use `Literal` for constrained choices (enums).
- Structured outputs are the foundation of reliable agentic pipelines.

3.3 Step 3: The ReAct Loop

The ReAct (Reason + Act) loop is the heart of every AI agent. It is a cycle where the model thinks, acts (calls a tool), observes the result, and repeats until the task is complete.

```python
"""
PHASE 1 - STEP 3: The ReAct Loop (Reason + Act)
Harness Pillar: Standard Workflows & Refinement Loops
Goal: Build the core agentic loop from scratch without frameworks.
"""

import math
import json
import anthropic
from dotenv import load_dotenv

load_dotenv()
client = anthropic.Anthropic()

# Tool definition (The ACI)
TOOLS = [{
    "name": "calculator",
    "description": "Perform mathematical calculations.",
    "input_schema": {
        "type": "object",
        "properties": {
            "expression": {"type": "string", "description": "Python math expression"}
        },
        "required": ["expression"]
    }
}]

def run_agent(user_query: str) -> str:
    messages = [{"role": "user", "content": user_query}]
    while True:
        response = client.messages.create(
            model="claude-opus-4-5-20251101",
```

```
            max_tokens=1024,
            tools=TOOLS,
            messages=messages,
        )
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            return response.content[-1].text

        if response.stop_reason == "tool_use":
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    # Execute tool (omitted for brevity)
                    result = "4"
                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": result,
                    })
            messages.append({"role": "user", "content": tool_results})
```

Listing 3.3: step3_react_loop.py

Key Takeaways

- The ReAct loop is a `while` loop keyed on `stop_reason`.
- Always append the assistant's full content to the message history to give it "memory."
- Always cap iterations to prevent runaway loops.

Chapter 4

Phase 2: ACI and Tool Engineering (The Hands Phase)

The goal of this phase is to build robust interfaces that agents can reliably use. We move from simple functions to tools with built-in safety guardrails and error-handling loops.

4.1 Step 4: Safe SQL Tool

This step demonstrates building a read-only SQL tool. We implement essential ACI guardrails: enforcing `SELECT`-only queries, injecting `LIMIT` clauses, and catching SQL errors to return them as structured JSON for self-correction.

```python
"""
PHASE 2 - STEP 4: ACI - Safe SQL Read Tool
Harness Pillar: ACI (Agent-Computer Interface)
Goal: Build a read-only SQL tool with safety guardrails.
"""

import sqlite3
import re
import json

MAX_ROWS = 50
FORBIDDEN_PATTERNS = [r"\bDROP\b", r"\bDELETE\b", r"\bINSERT\b", r"\bUPDATE\b"]

def safe_sql_query(query: str, db_path: str) -> dict:
    query = query.strip()
    # Guardrail: Only SELECT allowed
    if not query.upper().lstrip().startswith("SELECT"):
        return {"success": False, "error": "FORBIDDEN: Only SELECT allowed."}

    # Guardrail: Block mutation keywords
    for pattern in FORBIDDEN_PATTERNS:
        if re.search(pattern, query, re.IGNORECASE):
            return {"success": False, "error": f"FORBIDDEN: Disallowed keyword."}
```

```python

    # Guardrail: Inject LIMIT
    if "LIMIT" not in query.upper():
        query = f"{query.rstrip(';')} LIMIT {MAX_ROWS};"

    try:
        conn = sqlite3.connect(db_path)
        conn.row_factory = sqlite3.Row
        cur = conn.cursor()
        cur.execute(query)
        rows = [dict(row) for row in cur.fetchall()]
        conn.close()
        return {"success": True, "rows": rows}
    except sqlite3.Error as e:
        return {"success": False, "error": str(e), "hint": "Check column names."}
```

Listing 4.1: step4_sql_tool.py

Key Takeaways

- The ACI Observation Layer should be read-only by default to protect your data.
- Always return errors as structured JSON, not exceptions; the agent reads the error and self-corrects.
- Inject `LIMIT` automatically to prevent performance issues.

4.2 Step 6: Error-Handling Refinement

The refinement loop is what separates a fragile script from a resilient agent. By setting `is_error=True` and providing the full stack trace, we enable Claude to fix its own mistakes autonomously.

```python
"""
PHASE 2 - STEP 6: Error-Handling Refinement Loop
Harness Pillar: Standard Workflows & Refinement Loops
Goal: Feed tool errors back to the model so it can self-correct.
"""

import anthropic
import json

def run_self_correcting_agent(task: str):
    messages = [{"role": "user", "content": task}]
    while True:
        response = client.messages.create(
            model="claude-opus-4-5-20251101",
            max_tokens=2048,
            tools=TOOLS,
            messages=messages,
```

```
        )
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            return response.content[-1].text

        if response.stop_reason == "tool_use":
            tool_results = []
            for block in response.content:
                if block.type == "tool_use":
                    # Simulated execution with error
                    result = "NameError: name 'fib' is not defined"
                    is_error = True

                    tool_results.append({
                        "type": "tool_result",
                        "tool_use_id": block.id,
                        "content": result,
                        "is_error": is_error,  # Critical signal for self-correction
                    })
            messages.append({"role": "user", "content": tool_results})
```

Listing 4.2: step6_error_refinement.py

Key Takeaways

- `is_error=True` in the tool result is the signal that activates self-correction.
- Include the full error or stack trace in the tool result content.
- The refinement loop is essential for production agents operating in real-world, error-prone environments.

Chapter 5

Phase 3: Memory and Orchestration (The Brain Phase)

In this phase, we move beyond stateless interactions to managing context over long-running tasks. We explore how to load just-in-time information (RAG), manage token limits (Compaction), and enforce strict workflows (State Machines).

5.1 Step 7 & 8: RAG and Context Compaction

We treat the context window like RAM and external storage like a hard drive. Step 7 focuses on retrieving documentation on demand, while Step 8 introduces a method to summarize long histories to stay within token limits.

```python
"""
PHASE 3 - STEP 7 & 8: Memory - RAG + Context Compaction
Harness Pillar: Context Management
Goal: Just-in-time retrieval and proactive context management.
"""

import json
import anthropic

KNOWLEDGE_BASE = {"python_async": {"content": "asyncio enables concurrent code..."}}

def simple_retrieval(query: str):
    # Keyword-based retrieval (simplified)
    return [doc for doc in KNOWLEDGE_BASE.values() if any(word in doc['content'] for word in query.split())]

def compact_history(messages: list):
    """Summarize the conversation history down to bullet points."""
    history_text = "\n".join(f"{m['role'].upper()}: {m['content']}" for m in messages[1:-2])
    summary_response = client.messages.create(
        model="claude-opus-4-5-20251101",
        max_tokens=512,
```

```
        messages=[{"role": "user", "content": f"Summarize concisely:\n{history_text}"}],
    )
    summary = summary_response.content[0].text
    return [messages[0], {"role": "user", "content": f"[SUMMARY]\n{summary}"}, *messages[-2:]]
```

Listing 5.1: step7_8_rag_compaction.py

Key Takeaways

- NEVER stuff all documents into the context window upfront; retrieve on demand to save tokens and maintain focus.
- Estimate tokens before every API call and compact the history proactively when approaching limits.
- Always preserve the original first message and the most recent context during compaction.

5.2 Step 9: State Machines

A state machine forces an agent into programmatic, verifiable stages. This prevents the agent from skipping planning, rushing to action, or forgetting to validate its output.

```
"""
PHASE 3 - STEP 9: State Machine (Plan -> Execute -> Review)
Harness Pillar: Standard Workflows & Refinement Loops
Goal: Enforce a strict Plan -> Execute -> Review workflow.
"""

from enum import Enum
from dataclasses import dataclass

class State(Enum):
    PLAN = 1; EXECUTE = 2; REVIEW = 3; DONE = 4

@dataclass
class AgentContext:
    task: str; state: State = State.PLAN; plan: list = None

def run_state_machine(task: str):
    ctx = AgentContext(task=task)
    while ctx.state != State.DONE:
        if ctx.state == State.PLAN:
            ctx.plan = ["Step 1", "Step 2"] # Simplified planning
            ctx.state = State.EXECUTE
        elif ctx.state == State.EXECUTE:
            # Execute tools based on plan
            ctx.state = State.REVIEW
        elif ctx.state == State.REVIEW:
```

```
            # Separate judge call verifies results
            ctx.state = State.DONE
    return "Task Completed."
```

Listing 5.2: step9_state_machine.py

Key Takeaways

- A state machine removes ambiguity from the agentic process by enforcing verified stages.
- Use different system prompts for different roles (Planner, Executor, Judge) to maintain professional separation.
- The Review stage (LLM-as-judge) should ideally use a separate model call to avoid self-serving bias from the Executor.

Chapter 6

Phase 4: Production and Safety (The Harness Phase)

The final phase of becoming a Harness Engineer involves deploying agents safely and measuring their performance. We implement the complete safety harness: sandboxed computation, automated evaluation, and human approval gates.

6.1 Step 10: Sandboxed Code Execution

Executing agent-generated code requires an isolated environment. Anthropic's `code_execution` tool provides a secure, server-side container with popular data science libraries pre-installed.

```python
"""
PHASE 4 - STEP 10: Sandboxed Code Execution
Harness Pillar: ACI (Compute Layer)
Goal: Securely execute agent-generated Python code.
"""

import anthropic

def run_sandboxed_agent(task: str):
    response = client.beta.messages.create(
        model="claude-opus-4-5-20251101",
        max_tokens=4096,
        betas=["code-execution-2025-08-25"],
        tools=[{"type": "code_execution_20250825"}],
        system="You are a data analyst. Write and execute Python code.",
        messages=[{"role": "user", "content": task}],
    )
    return response.content[0].text
```

Listing 6.1: step10_sandboxed_agent.py

Key Takeaways

- Use isolated containers with no network access for agent code execution to prevent data exfiltration.
- Anthropic's sandbox comes with pre-installed data science tools like `pandas` and `matplotlib`.
- If running your own sandbox (e.g., Docker), always use `-network none` and `-read-only` flags.

6.2 Step 11: Evaluation Suite

Automated evaluation is critical for measuring agent performance. We use the "LLM-as-judge" pattern to grade agent outputs against a test dataset of tasks and requirements.

```python
"""
PHASE 4 - STEP 11: LLM-as-Judge Evaluation Suite
Harness Pillar: Evaluation & Guardrails
Goal: Build a pipeline that tests your agent and grades results.
"""

from pydantic import BaseModel
from typing import Literal

class EvalResult(BaseModel):
    verdict: Literal["pass", "fail"]
    score: float
    reasoning: str

def llm_judge(task: str, output: str, requirements: list) -> EvalResult:
    response = client.messages.parse(
        model="claude-opus-4-5-20251101",
        output_format=EvalResult,
        system="You are a strict technical evaluator.",
        messages=[{"role": "user", "content": f"Task: {task}\nOutput: {output}\nReqs: {requirements}"}],
    )
    return response.content[0].parsed
```

Listing 6.2: step11_evaluation_suite.py

Key Takeaways

- The evaluator model should be separate from the agent model to maintain objectivity.
- Use structured outputs (Pydantic) for the judge to ensure deterministic verdicts and parseable scores.
- Run evaluation suites in CI/CD before deploying any agent updates.

6.3 Step 12: Human-in-the-Loop (HITL)

For high-stakes actions like sending emails or mutating databases, a human approval gate is mandatory. The harness pauses execution and waits for a human decision (Approve, Reject, or Modify).

```python
"""
PHASE 4 - STEP 12: Human-in-the-Loop (HITL) Approval Gate
Harness Pillar: Evaluation & Guardrails
Goal: Pause the agent on high-stakes actions.
"""

HIGH_STAKES_TOOLS = {"send_email", "write_database"}

def run_hitl_agent(task: str):
    messages = [{"role": "user", "content": task}]
    # ... ReAct Loop ...
    if tool_name in HIGH_STAKES_TOOLS:
        print(f"[APPROVAL REQUIRED]: {tool_name}")
        decision = "approve" # Simulate human input
        if decision == "reject":
            # Return error to model
            pass
```

Listing 6.3: step12_hitl_agent.py

Key Takeaways

- Maintain an explicit allowlist of tools that require a permission bridge.
- The approval gate pauses the agentic loop rather than canceling it, allowing the human to course-correct the input.
- Real-world implementations often use Slack buttons, email approval links, or specialized web modals.